

Experiment No. 1

Aim: Implement Inferencing with Bayesian Network in Python

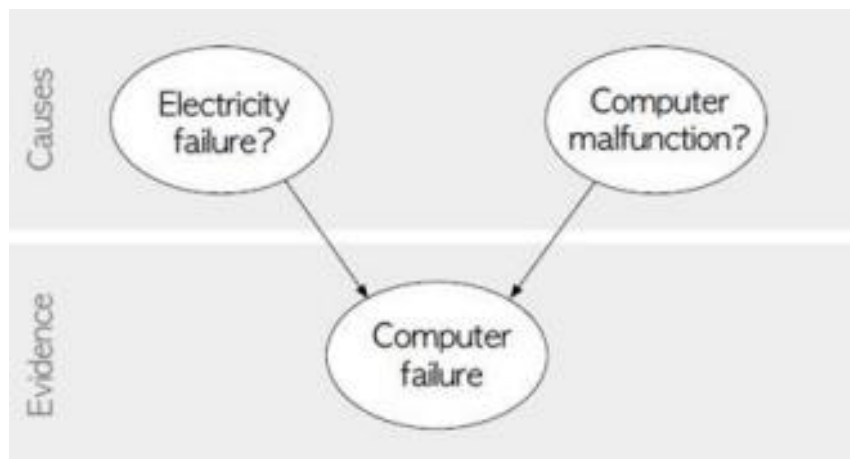
Theory:

A Bayesian network is a directed acyclic graph in which each edge corresponds to a conditional dependency, and each node corresponds to a unique random variable. Bayesian network consists of two major parts: a directed acyclic graph and a set of conditional probability distributions. The directed acyclic graph is a set of random variables represented by nodes. The conditional probability distribution of a node (random variable) is defined for every possible outcome of the preceding causal node(s).

A **Bayesian Network** (BN) is a probabilistic graphical model that represents a set of variables and their conditional dependencies using a directed acyclic graph (DAG). Each node in the graph represents a variable, and each directed edge represents a dependency between two variables. Bayesian Networks are extensively used in areas such as diagnostics, prediction, and decision-making under uncertainty.

For illustration, consider the following example. Suppose we attempt to turn on our computer, but the computer does not start (observation/evidence). We would like to know which of the possible causes of computer failure is more likely. In this simplified illustration, we assume only two possible causes of this misfortune: electricity failure and computer malfunction.

The corresponding directed acyclic graph is depicted in below figure.



Problem Statement:

To demonstrate the Bayesian Network, we'll use the well-known "Rain-Sprinkler-Grass" problem. The scenario involves three variables:

1. **Rain:** Whether it is raining or not.
2. **Sprinkler:** Whether the sprinkler is on or off.
3. **Grass Wet:** Whether the grass is wet or dry.

The relationships are as follows:

- Rain affects whether the sprinkler is turned on.
- Both rain and the sprinkler influence whether the grass is wet.

Input:

Step 1: Installing Necessary Libraries

```
import numpy as np
import pandas as pd
from pgmpy.models import BayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
```

Step 2: Defining the Structure of the Bayesian Network

```
model = BayesianNetwork([
('Rain', 'Sprinkler'),
('Rain', 'Grass Wet'),
('Sprinkler', 'Grass Wet')
])
```

Step 3: Defining the Conditional Probability Distributions

```
cpd_rain = TabularCPD(variable='Rain', variable_card=2, values=[[0.7], [0.3]])
cpd_sprinkler = TabularCPD(variable='Sprinkler', variable_card=2, values=[[0.6, 0.99], [0.4, 0.01]], evidence=['Rain'], evidence_card=[2])
cpd_grass_wet = TabularCPD(variable='Grass Wet', variable_card=2, values=[[1.0, 0.1, 0.1, 0.01], [0.0, 0.9, 0.9, 0.99]], evidence=['Sprinkler', 'Rain'], evidence_card=[2, 2])
```

Step 4: Adding CPDs to the Model

```
[7] # Adding CPDs to the model
    model.add_cpds(cpd_rain, cpd_sprinkler, cpd_grass_wet)

    # Verifying the model to ensure it's correctly defined
    if model.check_model():
        print("The Bayesian Network model is valid.")
    else:
        print("There is an error in the model.")
```

 The Bayesian Network model is valid.

Step 5: Performing Inference with Variable Elimination

```
infer = VariableElimination(model)
prob_grass_wet_given_sprinkler = infer.query(variables=['Grass Wet'], evidence={'Sprinkler': 1})
print("P(Grass Wet | Sprinkler=On):")
print(prob_grass_wet_given_sprinkler)
prob_rain_given_grass_wet = infer.query(variables=['Rain'], evidence={'Grass Wet': 1})
```

```
print("\nP(Rain | Grass Wet=Yes):")
print(prob_rain_given_grass_wet)
```

```
⇒ P(Grass Wet | Sprinkler=On):
+-----+-----+
| Grass Wet | phi(Grass Wet) |
+=====+=====+
| Grass Wet(0) | 0.0990 |
+-----+-----+
| Grass Wet(1) | 0.9010 |
+-----+-----+

P(Rain | Grass Wet=Yes):
+-----+-----+
| Rain | phi(Rain) |
+=====+=====+
| Rain(0) | 0.4825 |
+-----+-----+
| Rain(1) | 0.5175 |
+-----+-----+
```

Step 6: Advanced Inference - Marginal and Joint Distributions

```
marginal_rain = infer.query(variables=['Rain'])
print("\nMarginal Distribution P(Rain):")
print(marginal_rain)
```

```
⇒ Marginal Distribution P(Rain):
+-----+-----+
| Rain | phi(Rain) |
+=====+=====+
| Rain(0) | 0.7000 |
+-----+-----+
| Rain(1) | 0.3000 |
+-----+-----+
```

```
joint_rain_sprinkler = infer.query(variables=['Rain', 'Sprinkler'])
print("\nJoint Distribution P(Rain, Sprinkler):")
print(joint_rain_sprinkler)
```

```
⇒ Joint Distribution P(Rain, Sprinkler):
+-----+-----+-----+
| Rain | Sprinkler | phi(Rain,Sprinkler) |
+=====+=====+=====+
| Rain(0) | Sprinkler(0) | 0.4200 |
+-----+-----+-----+
| Rain(0) | Sprinkler(1) | 0.2800 |
+-----+-----+-----+
| Rain(1) | Sprinkler(0) | 0.2970 |
+-----+-----+-----+
| Rain(1) | Sprinkler(1) | 0.0030 |
+-----+-----+-----+
```

Conclusion: Thus, we have successfully understood Bayesian Network in Python and implemented with help of an Example